PADBen: Supplementary Appendix

Yiwei Zha[1], Rui Min[2], and Shanu Sushmita[2]

[1] Vrije Universiteit Amsterdam, De Boelelaan 1105, Amsterdam, The Netherlands
[2] Northeastern University, 360 Huntington Ave, Boston, MA 02115, United States

APPENDIX

The code and dataset are publicly available at: `https://github.com/JonathanZha47/PadBen-Paraphrase-Attack-Benchmark`

A Overall Data Processing

The experiment utilizes 16,233 human-authored sentences sourced from three established datasets following the pipeline shown in Figure 1:

- **MRPC** (Microsoft Research Paraphrase Corpus) [1]
- **HLPC** (Human-Like Paraphrase Corpus)[2]
- **PAWS** (Paraphrase Adversaries from Word Scrambling)[3]

A.1 MRPC Processing

The Microsoft Research Paraphrase Corpus (MRPC) [1] is a widely-used benchmark dataset for paraphrase detection, containing sentence pairs extracted from online news sources with human annotations indicating semantic equivalence. For dataset construction, only verified paraphrase pairs (`label == 1`) are extracted, utilizing `sentence1` as human original text and `sentence2` as human paraphrased text. This filtering ensures high-quality semantic equivalence relationships while maintaining the news domain characteristics.

A.2 PAWS Processing

The Paraphrase Adversaries from Word Scrambling (PAWS) dataset [3] is specifically designed to challenge paraphrase identification systems with adversarial examples. The PAWS-QQP version is adopted, utilizing the `labeled_final` subset and extracting only verified paraphrase pairs (`label == 1`), treating `sentence1` as human original text and `sentence2` as human paraphrased text.

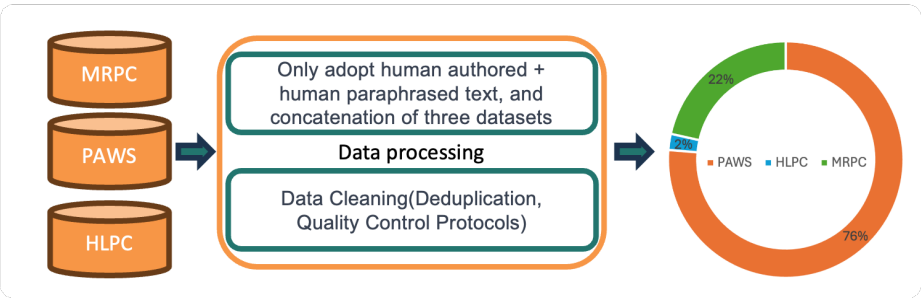

Fig. 1: The complete integration of HLPC, MRPC, and PAWS datasets follows a systematic pipeline that encompasses data loading, standardization, quality control, and deduplication.

A.3 HLPC Processing

The Human & LLM Paraphrase Collection (HLPC) [2] is a comprehensive dataset aggregating paraphrase data from multiple established sources. The generated variants of HLPC are considered outdated since they mainly use GPT-2-XL as main language model. Hence, only `originalSentence1` and `originalSentence2` are utilized to extract high-quality human paraphrase pairs.

A.4 Preprocessing

Given the potential overlap between datasets (particularly between HLPC and MRPC), a systematic deduplication process is implemented to prevent data leakage.

Quality Control Protocols

1. **Text Length Validation**: Remove entries with texts shorter than 10 characters or longer than 1000 characters
2. **Encoding Validation**: Ensure proper UTF-8 encoding and remove entries with encoding issues

Similarity-Based Duplicate Detection TF-IDF vectorization combined with cosine similarity is employed to identify near-duplicate content across the combined dataset:

$$\text{Similarity}(t_i, t_j) = \frac{\mathbf{v}_i \cdot \mathbf{v}_j}{|\mathbf{v}_i||\mathbf{v}_j|} \tag{1}$$

where $\mathbf{v}_i$ and $\mathbf{v}_j$ are TF-IDF vectors for texts t_i and t_j.

Algorithm 1 Deduplication Process

1: Compute TF-IDF vectors for all `human_original_text` entries
2: Calculate pairwise cosine similarities
3: **for** each text pair (t_i, t_j) where Similarity$(t_i, t_j) > \theta$ **do**
4: Identify as potential duplicate
5: Retain entry with higher dataset priority: PAWS $>$ MRPC $>$ HLPC
6: Mark duplicate for removal
7: **end for**
8: Remove identified duplicates from combined dataset

The threshold θ is set to 0.85 to ensure all adopted human-authored texts are unique.

B Intrinsic Mechanisms of Paraphrase Attacks

B.1 Experiment 1: Semantic Equivalence versus Paraphrasing in Representation Space

Research Objective In this experiment, the primary goal is to verify whether Hypothesis 1 holds: that paraphrasing induces a distinct semantic transformation, differing from text generated via "semantic equivalence" prompting. This is assessed by comparing the embedding spaces of LLM-paraphrased and LLM-generated texts.

Data Preparation Human-authored original sentences were collected from three established paraphrase datasets, following the preprocessing pipeline shown in Figure 1.

After applying quality control and deduplication procedures (detailed in Appendix A), 16,233 unique human-authored sentences were obtained that serve as the foundation for generating the other two text categories.

LLM-Generated Text Creation Using the 16,233 human-authored sentences as source material, semantically equivalent sentences were generated through LLM prompting. Unlike paraphrasing, this generation process aims to preserve the original meaning and structure without explicit rewording. The following semantic equivalence prompt was employed:

```
Given the following sentence, generate a new sentence that is semantically
equivalent, preserving the original meaning and structure as closely
as possible. Do not paraphrase or reword unnecessarily.
{text}
Generated sentence:
```

LLM-Paraphrased Text Creation From the same 16,233 human-authored sentences, paraphrased versions were generated using explicit paraphrasing instructions. The paraphrasing prompt was:

```
Please paraphrase the following text while maintaining its original
meaning:
{text}
Paraphrased text:
```

Experimental Significance The experimental framework addresses two critical hypotheses:

- **H1**: If LLM Generated ≈ LLM Paraphrased semantically, then paraphrase attacks exploit the same semantic space as original generation
- **H2**: If LLM Generated ≠ LLM Paraphrased, paraphrases create a distinct "attack space" requiring separate detection strategies

Embedding Generation BGE-M3 Model Configuration
The BGE-M3 (BAAI General Embedding Model) is employed for generating high-dimensional semantic representations. The embedding generation process is illustrated in Algorithm 2.

Algorithm 2 Embedding Generation Process

```
Initialize OpenAI client with BGE-M3 endpoint
Input: Text corpus T = {T_human, T_LLM, T_para}
for each text category t ∈ {human, LLM, para} do
    for each sentence s ∈ T_t do
        e_s ← BGE-M3(s) {Generate embedding vector}
    end for
    Store embeddings as E_t = {e_s : s ∈ T_t}
end for
Output: Embedding matrices {E_human, E_LLM, E_para}
```

Distance Analysis Distance Metrics
Three complementary distance metrics are computed between embedding pairs to capture different aspects of semantic similarity:

Cosine Similarity

$$d_{\text{cosine}}(\mathbf{u}, \mathbf{v}) = 1 - \frac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}||\mathbf{v}|} = 1 - \frac{\sum_{i=1}^{n} u_i v_i}{\sqrt{\sum_{i=1}^{n} u_i^2} \sqrt{\sum_{i=1}^{n} v_i^2}} \tag{2}$$

where $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$. Range: $[0, 1]$ where 0 indicates identical direction and 1 indicates orthogonality.

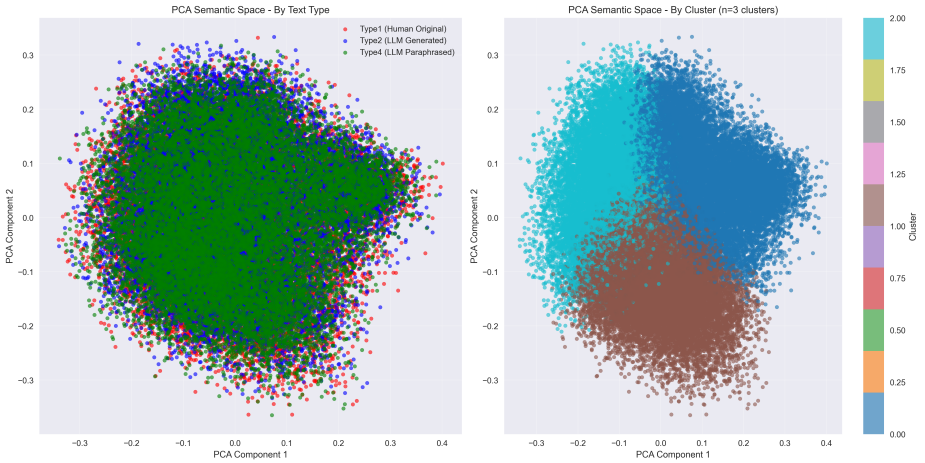

Fig. 2: PCA projection of semantic space (left) and K-means clustering results (right, $k = 3$). Despite measurable distance differences (Table **??**), text categories show substantial overlap in 2D projection, indicating that distinguishing information exists in higher dimensions.

Table 1: Full Table of Table **??**, with the addition of Euclidean Distance and Manhattan Distance

Comparison	**Cosine**	**Eucl.**	**Manh.**
Human ↔ LLM-Gen.	0.195	0.605	15.318
Human ↔ LLM-Para.	**0.068**	**0.355**	**8.991**
LLM-Gen. ↔ LLM-Para.	0.214	0.637	16.129

Euclidean Distance

$$d_{\text{euclidean}}(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{i=1}^{n}(u_i - v_i)^2} = |\mathbf{u} - \mathbf{v}|_2 \tag{3}$$

Manhattan Distance

$$d_{\text{manhattan}}(\mathbf{u}, \mathbf{v}) = \sum_{i=1}^{n}|u_i - v_i| = |\mathbf{u} - \mathbf{v}|_1 \tag{4}$$

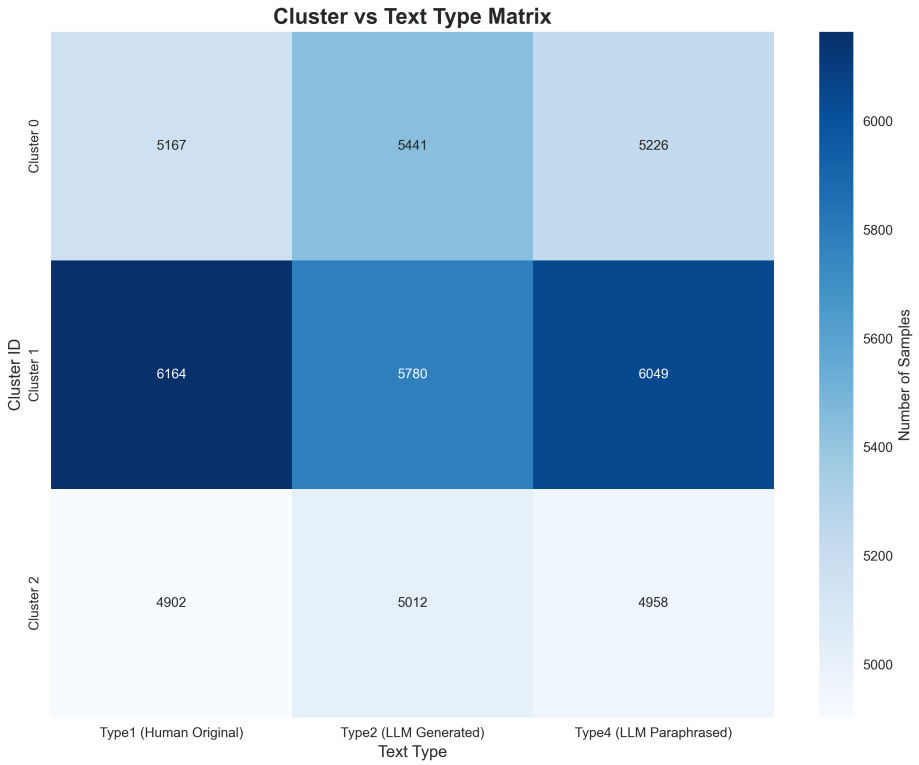

Fig. 3: Clustering label distribution across human-authored, LLM-generated, and LLM-paraphrased text. The matrix reveals that low-dimensional representation space makes it difficult to distinguish between the three text types.

Pairwise Distance Computation For each distance metric d, average distances between text type pairs are computed:

$$D^{(d)}_{H,L} = \frac{1}{|E_H| \cdot |E_L|} \sum_{e_h \in E_H} \sum_{e_l \in E_L} d(e_h, e_l) \tag{5}$$

$$D^{(d)}_{H,P} = \frac{1}{|E_H| \cdot |E_P|} \sum_{e_h \in E_H} \sum_{e_p \in E_P} d(e_h, e_p) \tag{6}$$

$$D^{(d)}_{L,P} = \frac{1}{|E_L| \cdot |E_P|} \sum_{e_l \in E_L} \sum_{e_p \in E_P} d(e_l, e_p) \tag{7}$$

where H, L, and P denote human-authored, LLM-generated, and paraphrased text, respectively(shown at Table 1).

Semantic Space Exploration

Dimensionality Reduction via PCA Principal Component Analysis (PCA) is applied to project the high-dimensional BGE-M3 embeddings (1024 dimensions) into 2D visualization space. Figure 2 shows the PCA visualization.

Unsupervised Clustering via KMeans K-Means clustering with $k = 3$ clusters is applied to the combined embedding space to identify natural semantic groupings. Figure 2 (right) shows the KMeans clustering visualization, and Figure 3 shows the detailed label distribution.

B.2 Experiment 2: Iterative Paraphrasing in Representation Space

Research Objective This experiment investigates **Hypothesis 2**: Iterative paraphrasing makes text become more coherent, deviating from common LLM-generated patterns and moving closer to human-authored texts. Semantic drift is analyzed through multiple iterations of paraphrasing to understand how text evolves in semantic space over successive transformations.

Data Preparation The experiment randomly samples 1000 human-authored texts from the combined dataset (detailed in Appendix A).

Experiment Procedure

Representation Space Choice For each iteration, two complementary representations are extracted:

1. **Hidden States**: Last-layer hidden states from Qwen3-4B-Instruct with mean pooling across sequence length
2. **Semantic Embeddings**: BGE-M3 embeddings via Novita AI API for semantic space analysis

Distance Analysis Two complementary distance analyses are conducted to examine semantic drift under iterative paraphrasing:

Analysis 1: Progressive Semantic Drift. The distance between consecutive iterations is measured by computing distances between iteration i and iteration $i + 1$ for both human-authored and LLM-generated text that have undergone 1–10 paraphrasing iterations. The result table can be found in Table **??**.

Analysis 2: Cross-Category Semantic Distance. Semantic distances are measured between original human-authored text and paraphrased human-authored text (iterations 1–10), as well as between original LLM-generated text and paraphrased human-authored text (iterations 1–10). The result table can be found in Table 3.

For both analyses, three complementary distance metrics are employed to capture different aspects of semantic dissimilarity: cosine similarity, Euclidean distance, and Manhattan distance (see Equations 2, 3, and 4).

Table 2: Cosine similarity: (1) single iteration of paraphrased human-authored text versus 2–10 iterations; (2) single iteration of paraphrased LLM-generated text versus 2–10 iterations, based on BGE-M3 embedding and Qwen3-4B final hidden state. H. indicates human-authored text, L. indicates LLM-generated text.

Repr.	**Type**	**Iteration**				
		2	**4**	**6**	**8**	**10**
BGE Emb.	H	0.048	0.072	0.083	0.092	0.100
	L	0.047	0.068	0.077	0.087	0.091
Hid. Stat.	H	0.012	0.022	0.015	0.017	0.019
	L	0.011	0.014	0.015	0.017	0.018

Analysis 1 uses centroid-based distance to measure population-level drift. For each iteration i and text type t, population centroids are computed:

$$\mathbf{c}_{i,t} = \frac{1}{N} \sum_{j=1}^{N} \mathbf{e}_{i,t,j} \tag{8}$$

where $\mathbf{e}_{i,t,j}$ represents the embedding of sample j at iteration i for text type t. Sequential distance analysis then tracks semantic drift between consecutive iterations:

$$\Delta d_{i \to i+1} = d(\mathbf{c}_{i,t}, \mathbf{c}_{i+1,t}) \tag{9}$$

Analysis 2 employs the pairwise distance calculation described in Equation 5, measuring distances between the reference texts (human-authored original and LLM-generated original) and iteratively paraphrased human-authored text at each iteration level.

PCA Trajectory Analysis PCA is applied to project high-dimensional embeddings into 2D visualization space. This trajectory analysis tracks how text representations evolve through successive paraphrasing iterations.

A 5-iteration analysis was initially conducted to identify semantic drift patterns. However, the resulting trajectories did not reveal sufficiently clear patterns to draw robust conclusions about long-term semantic behavior (Figure 4). Consequently, the analysis was extended to 10 iterations, which provided more definitive evidence of semantic drift trajectories and convergence patterns.

The PCA trajectory analysis follows the procedure outlined in Algorithm 3:

Table 3: Semantic distance between two text types: (1) human-authored text versus 1–10 iterations of paraphrased human-authored text; (2) LLM-generated text versus 1–10 iterations of paraphrased human-authored text.

Text Type	Iter.	Cosine Distance	Euclidean Distance
Human-Authored	1	0.064	0.342
	2	0.085	0.394
	3	0.099	0.426
	4	0.107	0.443
	5	0.118	0.465
	6	0.122	0.472
	7	0.125	0.478
	8	0.128	0.484
	9	0.129	0.486
	10	0.134	0.494
LLM-Generated	1	0.700	1.182
	2	0.698	1.180
	3	0.696	1.179
	4	0.697	1.180
	5	0.696	1.179
	6	0.697	1.179
	7	0.697	1.180
	8	0.699	1.181
	9	0.697	1.179
	10	0.698	1.180

Algorithm 3 PCA Trajectory Analysis

1: Collect all embeddings across iterations: $E = \{\mathbf{e}_{i,t,j} : i \in [1, N_{iter}], t \in \{\text{type1}, \text{type2}\}, j \in [1, N_{samples}]\}$
2: Standardize features: $\tilde{E} = \text{StandardScaler}(E)$
3: Apply PCA: $E_{\text{PCA}} = \text{PCA}_{n=2}(\tilde{E})$
4: Compute iteration centroids in PCA space: $\mathbf{c}_{i,t}^{\text{PCA}} = \frac{1}{N} \sum_{j=1}^{N} \mathbf{e}_{i,t,j}^{\text{PCA}}$
5: Track centroid trajectories: $\mathcal{T}_t = \{\mathbf{c}_{i,t}^{\text{PCA}} : i \in [1, N_{iter}]\}$

To quantify the magnitude of semantic drift, the total Euclidean displacement of centroids across iterations is computed:

$$\text{Total Drift}_t = \sum_{i=1}^{N_{iter}-1} ||\mathbf{c}_{i+1,t}^{\text{PCA}} - \mathbf{c}_{i,t}^{\text{PCA}}||_2 \tag{10}$$

Figure 5 visualizes the resulting trajectories in PCA space, where each point represents the centroid of all samples at a given iteration, and connecting lines trace the semantic evolution path.

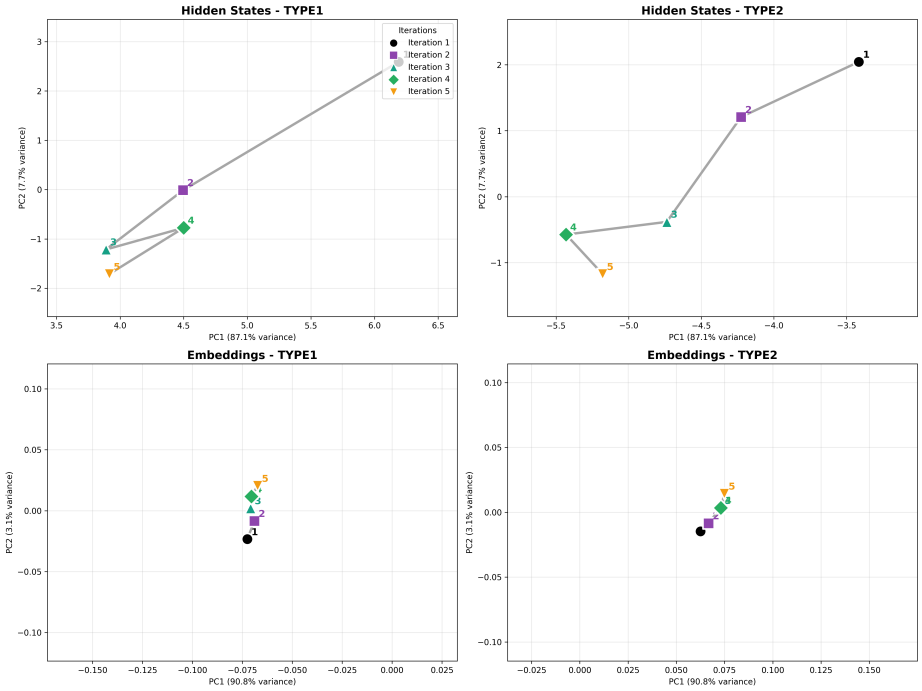

Fig. 4: The 5-iteration centroid trajectories under PCA ($n = 2$). Top: Hidden state space (left: human-origin, right: LLM-origin). Bottom: Embedding space.

Detailed Interpretation of Figure 5. The 10-iteration centroid trajectories reveal four key patterns:

1. **Universal directional drift (embedding space, bottom row).** In BGE-M3 embedding space, centroids from both human-origin and LLM-origin texts move consistently in similar directions across iterations. The trajectories do not oscillate but show a steady, gradual displacement—consistent with Table **??**, which shows monotonically increasing cosine similarity from the human-authored reference. This uniform drift across origins suggests that the paraphrasing operation itself (regardless of source) pushes texts in a common direction in semantic embedding space.
2. **Oscillatory behavior in hidden states (top row).** Hidden-state centroids show a different pattern: an initial displacement in the first few iterations followed by oscillations around a fixed region rather than continued drift. This is consistent with Table **??**, where inter-iteration hidden-state distances plateau or fluctuate (values 0.012–0.022) without accumulating. The oscillatory behavior suggests generation-level features are not systematically altered by paraphrasing—the LLM paraphraser reorganizes surface text but does not consistently shift the statistical generation signature.

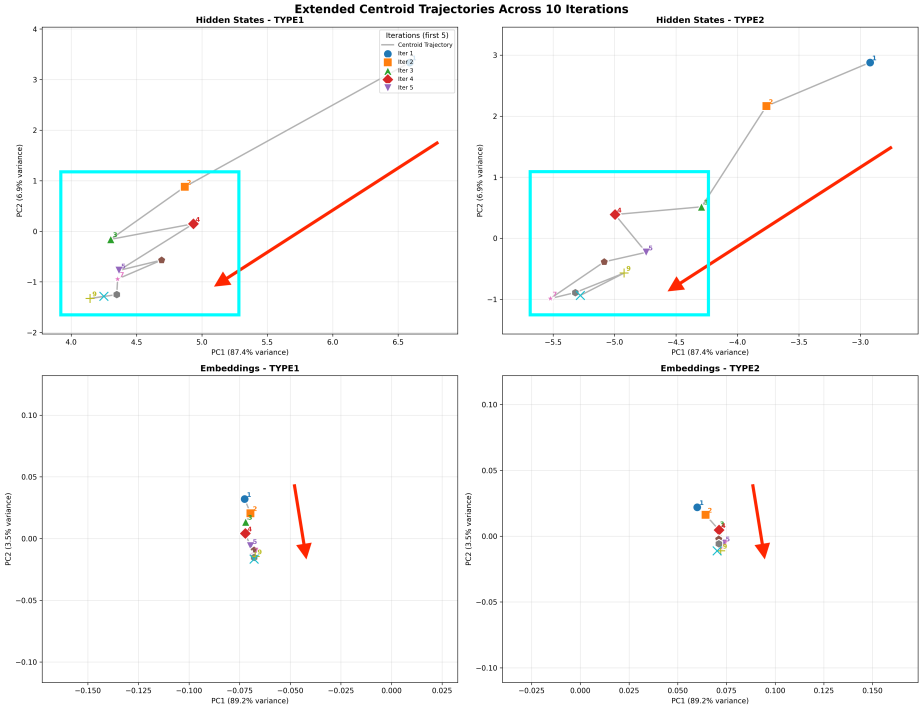

Fig. 5: Extended 10-iteration centroid trajectories showing semantic drift patterns. Top: Hidden state space (left: human-origin, right: LLM-origin). Bottom: Embedding space. Both origins move in parallel directions, with trajectories converging toward overlapping regions in later iterations.

3. **Trajectory convergence toward overlapping regions.** By iteration 8–10, the PCA centroids of human-origin and LLM-origin trajectories approach overlapping regions in both hidden-state and embedding spaces. This convergence in low-dimensional PCA space motivates the design of the benchmark tasks: if trajectories from different origins visually converge, distinguishing source origin (Task 3) or iteration depth (Task 4) becomes increasingly difficult for detectors relying on embedding-space signals.
4. **Asymmetry between origins.** While trajectories converge, the starting positions differ substantially. Human-origin centroids begin in the upper portion of PCA space; LLM-origin centroids begin in a distinct cluster. The convergence therefore requires traversal of the full distance between these starting regions over 10 iterations—a distance not fully closed in 1–3 iterations, explaining why Task 5 (human original vs. 3-iteration laundered AI) remains tractable: at 3 iterations, the LLM-origin trajectory has not yet fully overlapped with the human-origin region.

C Detailed Methodology

C.1 Dataset Preparation

The benchmark builds upon the human-authored sentences (Type 1) and human paraphrases (Type 3) from three established datasets: MRPC, HLPC, and PAWS. The detailed preprocessing pipeline, including deduplication and quality control procedures, is described in Appendix A. This foundation provides 16,233 unique human-authored sentences and human-paraphrased human texts, from which the remaining text types (Type 2, 4, 5) are systematically generated using the pipeline described in Appendix C.3.

C.2 Text Type Taxonomy

A five-category taxonomy is established to systematically analyze different text generation and paraphrasing patterns:

- **Type 1**: Human original text — authentic human-authored sentences
- **Type 2**: LLM-generated text — synthetically generated content maintaining semantic equivalence to Type 1 (generated by sentence completion method)
- **Type 3**: Human-paraphrased human original text — human-authored paraphrases of Type 1 sentences
- **Type 4**: LLM-paraphrased human original text — machine-generated paraphrases of Type 1 sentences
- **Type 5**: LLM-iteratively-paraphrased LLM-generated text — machine-generated paraphrases of Type 2 sentences with multiple iteration levels

C.3 Data Generation Pipeline

Technical Architecture The data generation system employs a modular, configuration-driven architecture with model selection optimized for each text type's specific requirements. The use of multiple models for paraphrasing (Type 4 and 5) is a deliberate choice to create a diverse dataset that is not biased toward the stylistic quirks of a single paraphraser.

The pipeline implements three sequential generation modules:

1. **Type 2 Generation Module**: Sentence completion-based text synthesis using **Google Gemini-2.5-Pro**
2. **Type 4 Generation Module**: Multiple-model paraphrasing combining DIPPER paraphraser [4], Gemini-2.5-Pro with prompt-based instructions, and LLaMA-3-8B paraphrase fine-tuned models [5]
3. **Type 5 Generation Module**: Iterative paraphrasing using the same multi-model approach as Type 4

Type 2 Generation: Sentence Completion Method A **sentence completion approach** is implemented for Type 2 text generation, designed to produce text that is contextually grounded in the original human sentence while allowing for natural, unconstrained continuation. The process involves:

1. **Keyword Extraction**: Using SpaCy's named entity recognition and dependency parsing to identify salient keywords from Type 1 sentences
2. **Prefix Extraction**: Extracting the first 20% of tokens from the original sentence as contextual seed
3. **Length Constraints**: Computing target length parameters with ±20% tolerance

Generation Prompt Template

```
Continue this text naturally and coherently:
"{sentence_prefix}"
Requirements:
- Target length: ~{target_length} characters total
- Maximum length: {max_length} characters
- Keywords to include: {keywords}
- Write in a natural, fluent style
Return ONLY the completed text with no labels, quotes, explanations, or
alternatives.
Completion:
```

Type 4 Generation: Direct Paraphrasing Type 4 generation employs **prompt-based paraphrasing** using carefully engineered instructions, prioritizing semantic preservation while encouraging significant lexical and syntactic variation.

Paraphrasing Prompt Template

```
Please paraphrase the following text while maintaining its original
meaning:
{text}
Paraphrased text:
```

Type 5 Generation: Iterative Paraphrasing The Type 5 module implements **multi-iteration paraphrasing** of Type 2 texts. Two levels are supported: 1 and 3 iterations, where each iteration applies the paraphrasing prompt to the output of the previous one.

Iteration Control Mechanisms:

- **Temperature Scaling**: Base temperature (0.8) increases by 0.1–0.15 per iteration level to enhance diversity
- **Convergence Detection**: Automatic termination when consecutive iterations achieve >95% similarity
- **Length Tolerance**: Expanded to ±40% to accommodate cumulative variation across iterations

C.4 Data Quality Assessment

Three complementary quality metrics are employed: **Jaccard similarity**, **perplexity**, and **self-BLEU** scores.

Jaccard Similarity Analysis: The inter-type similarity matrix reveals expected semantic relationships across text types. Human original text (Type 1) demonstrates highest similarity with human paraphrases (Type 3, Jaccard = 0.798), confirming semantic preservation in human paraphrasing. Notably, iterative paraphrasing exhibits controlled diversity: Type 5 first-iteration maintains reasonable similarity with its source (0.469 with Type 2), while third-iteration paraphrasing shows increased lexical divergence (0.423 with Type 2).

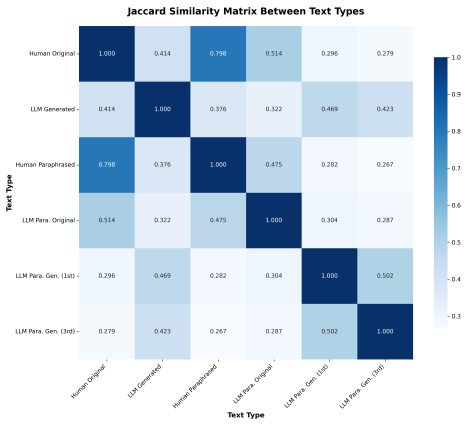

Fig. 6: Jaccard similarity matrix between different text types.

Perplexity Evaluation: Text predictability is assessed using perplexity scores computed with GPT-2-XL and LLaMA-2-7B as reference models. The quality metrics reveal consistent patterns across both evaluation models (Table 4). LLM-generated text (Type 2) exhibits the lowest perplexity scores (GPT-2-XL: 77.84, LLaMA-2-7B: 42.61), while human original and paraphrased texts consistently demonstrate higher perplexity scores, suggesting greater linguistic variability.

Self-BLEU Assessment: Self-BLEU scores (Table 4) demonstrate appropriate diversity levels across all text types, with iterative paraphrasing achieving maximum diversity (Type 5-3rd: 0.170).

Comparison with RAID: PADBen exhibits substantially lower self-BLEU scores (0.222 vs 13.7), indicating approximately 62× higher intra-type diversity. Using GPT-2-XL, PADBen achieves 4.1× higher perplexity scores (97.82 vs 23.8), while LLaMA-2-7B evaluation shows a 7.0× difference (46.46 vs 6.61).

Table 4: Quality metrics across text types.

Text Type	PPL-G2X	PPL-L7B	sBLEU
Type 1	106.78	49.57	0.268
Type 2	77.84	42.61	0.242
Type 3	107.47	49.45	0.275
Type 4	85.90	39.63	0.196
Type 5-1st	99.63	47.29	0.179
Type 5-3rd	109.32	50.23	0.170
Average	**97.82**	**46.46**	**0.222**

Table 5: Quality metrics comparison between PADBen and RAID datasets (average values).

Dataset	Self-BLEU	PPL-G2X	PPL-L7B
PADBen	0.222	97.82	46.46
RAID	13.7	23.8	6.6

C.5 Dataset Statistics

The final dataset comprises **16,232 sentence groups** across three source datasets, each containing all five text types.

C.6 Detailed Task Introduction

Task 1: Paraphrase Source Attribution Objective: Evaluate detectors' ability to distinguish between human and machine paraphrasing without access to original source text.

Data Configuration: Utilize Type 3 (human-paraphrased) and Type 4 (LLM-paraphrased) texts. Human paraphrases are labeled as 0, LLM-generated paraphrases as 1.

Detection Challenge: Without reference to original text, detectors must rely solely on intrinsic linguistic markers to differentiate human and machine paraphrasing strategies.

Task 2: General Text Authorship Detection Objective: Assess baseline detection performance distinguishing original human-authored text from LLM-generated content.

Data Configuration: Utilize Type 1 (human original) and Type 2 (LLM-generated) texts. Human-authored content is labeled as 0, LLM-generated text as 1.

Task 3: AI Text Laundering Detection Objective: Evaluate detectors' resilience against AI text laundering through paraphrasing attacks.

Data Configuration: Utilize Type 4 (LLM-paraphrased human text) and Type 5-1st (single-iteration LLM-paraphrased LLM text). LLM-paraphrased human content is labeled as 0, laundered AI text as 1.

Detection Challenge: Both texts have undergone identical machine transformation, requiring determination of which originated from human versus LLM sources before paraphrasing.

Task 4: Iterative Paraphrase Depth Detection Objective: Assess detector ability to distinguish between shallow and deep iterative paraphrasing attacks.

Data Configuration: Utilize Type 5-1st (single-iteration) and Type 5-3rd (triple-iteration) paraphrased LLM text. Shallow paraphrasing (1 iteration) is labeled as 0, deep paraphrasing (3 iterations) as 1.

Task 5: Paraphrase Attack Detection Objective: Evaluate detector resilience in the ultimate end-to-end evasion scenario.

Data Configuration: Utilize Type 1 (human original) and Type 5-3rd (triple-iteration LLM-paraphrased LLM text). Human text is labeled as 0, deeply laundered AI text as 1.

Detection Challenge: This is the benchmark's final stress test, requiring detectors to identify highly subtle, persistent machine-generation artifacts that have survived multiple layers of transformation.

D Detailed Task Data Setup

Algorithm 4 Single-Sentence Exhaustive Method

Input: Dataset D with n samples, Task specification (T_A, T_B)
Initialize: $samples \leftarrow \emptyset$
for each sample $s_i \in D$ **do**
    Extract text $text_A \leftarrow s_i[T_A]$
    Extract text $text_B \leftarrow s_i[T_B]$
    Create sample: $(idx = 2i, sentence = text_A, label = 0)$
    Create sample: $(idx = 2i + 1, sentence = text_B, label = 1)$
end for
Shuffle sample indices randomly
Output: Dataset of size $2n$ with balanced 50-50 label distribution

Five distinct experimental settings, illustrated in Fig 7, systematically vary data utilization strategies and task formulations to provide robust evaluation frameworks.

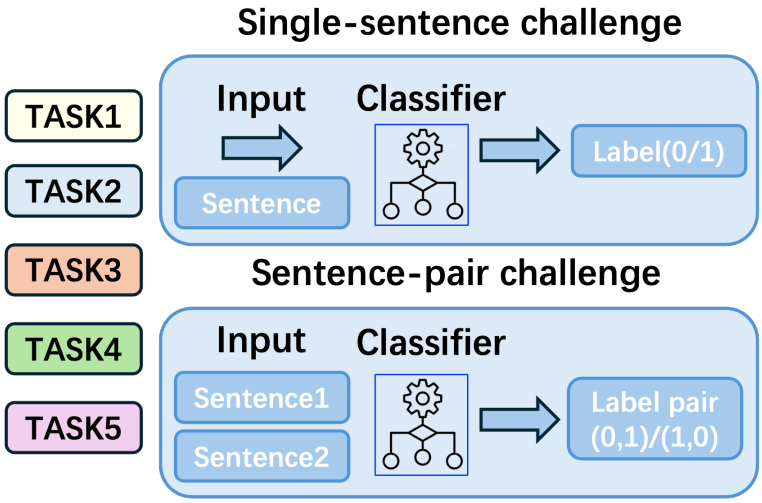

Fig. 7: Two evaluation challenges: single-sentence classification and sentence-pair recognition. All five tasks are transformed into these two challenge formats.

Rationale for Five-Setting Framework. The framework addresses fundamental limitations through systematic variation of three critical dimensions:
(1) **Data Utilization Strategy**: Exhaustive vs. sampling approaches to control semantic repetition;
(2) **Label Distribution**: Balanced vs. imbalanced scenarios to test base rate sensitivity;
(3) **Task Formulation**: Absolute vs. comparative classification paradigms.

D.1 Setting 1: Single-Sentence Exhaustive Method

The exhaustive method implements a comprehensive data utilization strategy where all available instances from both relevant text types are used to create the maximum possible dataset size. Algorithm 4 shows the technical specifics.

Characteristics. Dataset size: 2× original size (e.g., 16,233 → 32,466 samples); Label distribution: Fixed 50-50 balance; Data utilization: Exhaustive use of all available instances.

D.2 Settings 2–4: Single-Sentence Sampling Method

The sampling method addresses evaluation validity concerns by implementing **controlled semantic exposure**. Algorithm 5 provides technical details.

Distribution Settings. **Setting 2 (30-70):** Sampling probability $p = 0.3$, focusing on imbalanced dataset performance. **Setting 3 (50-50):** Sampling probability $p = 0.5$, enabling direct comparison with exhaustive method. **Setting 4 (80-20):** Sampling probability $p = 0.8$, testing detector robustness with majority LLM-generated content.

Algorithm 5 Single-Sentence Sampling Method

Input: Dataset D with n samples, Task (T_A, T_B), sampling ratio p
Initialize: $samples \leftarrow \emptyset$, random seed
for each sample $s_i \in D$ **do**
  Generate random value $r \sim \text{Uniform}(0, 1)$
  if $r < p$ **then**
    Select $text \leftarrow s_i[T_B]$, $label \leftarrow 1$
  else
    Select $text \leftarrow s_i[T_A]$, $label \leftarrow 0$
  end if
  Create sample: $(idx = i, sentence = text, label = label)$
  $samples \leftarrow samples \cup \{(i, text, label)\}$
end for
Output: Dataset of size n with target label distribution

D.3 Setting 5: Sentence-Pair Recognition

Sentence-pair recognition addresses a fundamental limitation in current AI text detection evaluation. Pair-wise evaluation better mirrors real-world scenarios where humans and detectors must choose between alternatives of unknown provenance.

Algorithm 6 Sentence-Pair Recognition Challenge

Input: Dataset D with n samples, Task (T_A, T_B)
Initialize: $pairs \leftarrow \emptyset$
for each sample $s_i \in D$ **do**
  Extract $sentence_A \leftarrow s_i[T_A]$
  Extract $sentence_B \leftarrow s_i[T_B]$
  Generate random bit $flip \sim \text{Bernoulli}(0.5)$
  if $flip = 0$ **then**
    $pair \leftarrow [sentence_A, sentence_B]$, $labels \leftarrow [0, 1]$
  else
    $pair \leftarrow [sentence_B, sentence_A]$, $labels \leftarrow [1, 0]$
  end if
  Create sample: $(idx = i, sentence\_pair = pair, label\_pair = labels)$
  $pairs \leftarrow pairs \cup \{(i, pair, labels)\}$
end for
Output: Dataset of n sentence pairs with randomized order

Table 6: Comparative analysis of the five evaluation settings.

Aspect	**Exhaustive**	**30-70**	**50-50**	**80-20**	**Sentence-Pair**
Dataset Size	$2n$	n	n	n	n
Label Distribution	50-50	30-70	50-50	80-20	Balanced pairs
Semantic Repetition	Present	Eliminated	Eliminated	Eliminated	Eliminated
Data Utilization	Complete	Sampled	Sampled	Sampled	Complete per pair
Evaluation Type	Absolute	Absolute	Absolute	Absolute	Comparative
Bias Concerns	Semantic	Distribution	Minimal	Distribution	Positional
Research Focus	Max performance	Minority class	Balanced	Majority class	Comparative

Table 7: Zero-shot Detectors Performance Summary

MODEL	CHALLENGE	METHOD	Task 1				Task 2				Task 3				Task 4				Task 5			
			AUC	T1	T5	T10	AUC	T1	T5	T10	AUC	T1	T5	T10	AUC	T1	T5	T10	AUC	T1	T5	T10
BINOCULAR	sentence-pair		0.399	0.003	0.023	0.050	0.457	0.007	0.037	0.081	0.505	0.008	0.046	0.099	0.498	0.011	0.052	0.106	0.457	0.006	0.032	0.070
	single-sentence	exhaustive	0.439	0.011	0.046	0.080	0.579	0.029	0.106	0.184	0.500	0.008	0.050	0.101	0.486	0.012	0.050	0.098	0.428	0.009	0.047	0.087
	single-sentence	sampling_30%	0.437	0.010	0.046	0.081	0.575	0.031	0.109	0.188	0.495	0.008	0.050	0.096	0.486	0.010	0.051	0.098	0.429	0.008	0.050	0.090
	single-sentence	sampling_50%	0.443	0.011	0.047	0.084	0.583	0.032	0.112	0.194	0.499	0.008	0.051	0.102	0.487	0.012	0.053	0.099	0.434	0.009	0.051	0.092
	single-sentence	sampling_80%	0.453	0.010	0.049	0.087	0.589	0.034	0.116	0.195	0.507	0.007	0.058	0.106	0.487	**0.016**	**0.060**	0.104	0.439	0.012	0.053	0.092
FAST_DETECT_GPT	sentence-pair		0.638	**0.027**	**0.115**	0.204	0.787	0.086	0.249	0.369	0.503	0.012	0.055	0.108	0.476	0.005	0.036	0.081	0.606	0.023	0.090	0.165
	single-sentence	exhaustive	0.573	0.006	0.044	0.099	0.665	0.010	0.075	0.149	0.504	0.011	0.051	0.103	0.488	0.009	0.051	0.099	0.568	0.006	0.047	0.103
	single-sentence	sampling_30%	0.576	0.006	0.046	0.097	0.666	0.009	0.078	0.154	0.502	0.011	0.049	0.096	0.490	0.007	0.046	0.091	0.572	0.005	0.045	0.100
	single-sentence	sampling_50%	0.581	0.005	0.045	0.098	0.672	0.010	0.078	0.153	0.505	0.010	0.051	0.101	0.491	0.008	0.049	0.095	0.577	0.005	0.047	0.102
	single-sentence	sampling_80%	0.587	0.004	0.040	0.092	0.675	0.008	0.076	0.151	0.514	0.007	0.045	0.100	0.488	0.011	0.049	0.095	0.578	0.005	0.048	0.103
GLTR	sentence-pair		0.429	0.007	0.043	0.082	0.436	0.006	0.045	0.086	0.529	0.011	0.060	0.116	0.514	**0.016**	**0.057**	**0.113**	0.482	0.012	0.054	0.108
	single-sentence	exhaustive	0.459	0.006	0.032	0.068	0.480	0.004	0.021	0.056	0.513	0.012	0.059	0.122	0.506	**0.013**	**0.057**	**0.113**	0.488	0.011	0.047	0.091
	single-sentence	sampling_30%	0.457	0.004	0.034	0.066	0.474	0.003	0.019	0.053	0.519	0.012	0.065	0.124	0.502	**0.014**	**0.056**	**0.109**	0.484	0.012	0.045	0.085
	single-sentence	sampling_50%	0.461	0.005	0.036	0.068	0.480	0.004	0.022	0.057	0.524	0.012	0.066	0.126	**0.507**	**0.015**	**0.059**	**0.114**	0.489	0.011	0.049	0.089
	single-sentence	sampling_80%	0.458	0.006	0.031	0.063	0.482	0.004	0.021	0.059	0.523	0.011	0.062	0.117	**0.509**	0.013	0.059	**0.111**	0.491	0.011	0.047	0.095
RADAR	sentence-pair		**0.728**	0.004	0.105	**0.246**	**0.910**	**0.142**	**0.566**	**0.809**	**0.748**	**0.054**	**0.234**	**0.372**	**0.526**	0.010	0.055	0.112	**0.909**	**0.140**	**0.542**	**0.808**
	single-sentence	exhaustive	**0.648**	**0.038**	**0.190**	**0.345**	**0.793**	**0.063**	**0.313**	**0.567**	**0.633**	**0.016**	**0.080**	**0.160**	**0.511**	0.010	0.052	0.104	**0.797**	**0.062**	**0.310**	**0.562**
	single-sentence	sampling_30%	**0.642**	**0.037**	**0.187**	**0.337**	**0.789**	**0.063**	**0.313**	**0.560**	**0.627**	**0.016**	**0.078**	**0.157**	**0.508**	0.010	0.051	0.103	**0.797**	**0.062**	**0.312**	**0.560**
	single-sentence	sampling_50%	**0.644**	**0.036**	**0.181**	**0.337**	**0.789**	**0.060**	**0.302**	**0.559**	**0.628**	**0.016**	**0.078**	**0.155**	0.506	0.010	0.051	0.102	**0.795**	**0.060**	**0.300**	**0.556**
	single-sentence	sampling_80%	**0.648**	**0.039**	**0.195**	**0.345**	**0.797**	**0.067**	**0.335**	**0.569**	**0.630**	**0.016**	**0.078**	**0.156**	0.508	0.010	0.051	0.102	**0.803**	**0.066**	**0.332**	**0.568**

Note: AUC = AUC-ROC, T1 = TPR@1%FPR, T5 = TPR@5%FPR, T10 = TPR@10%FPR. Best (**red bold**) and second-best (blue underlined) results are marked within each setup for each task and metric.

D.4 Summary

E Detailed Evaluation Settings

E.1 Zero-Shot Detectors: Complete Results

E.2 Evaluation Metrics

Area Under ROC Curve (AUROC): Measures the detector's ability to distinguish between human and AI-generated text across all classification thresholds. AUROC values range from 0.5 (random performance) to 1.0 (perfect classification).

TPR@1%FPR: True Positive Rate when False Positive Rate is constrained to 1%.

TPR@5%FPR: True Positive Rate when False Positive Rate is constrained to 5%.

TPR@10%FPR: True Positive Rate when False Positive Rate is constrained to 10%.

E.3 Zero-Shot Detectors Setup

Binoculars Configuration Model Configuration:

1. **Observer Model:** `tiiuae/falcon-7b`
2. **Performer Model:** `tiiuae/falcon-7b-instruct`
3. **Detection Mode:** Accuracy-optimized

GLTR Configuration Model Configuration:

1. **Primary Model:** GPT-2-large
2. **Analysis Window:** Full text sequences up to model maximum length

Fast-DetectGPT Configuration Model Configuration:

1. **Scoring Model:** `falcon-7b-instruct`
2. **Sampling Model:** Same as scoring model for efficiency
3. **Sampling Parameters:** 10,000 samples for probability estimation

The Fast-DetectGPT criterion is:

$$\text{Criterion} = \frac{\log p_\theta(x) - \mu_{\tilde{x}}}{\sigma_{\tilde{x}}} \tag{11}$$

where $\mu_{\tilde{x}}$ and $\sigma_{\tilde{x}}$ are sample statistics.

RADAR Configuration Model Configuration:

1. **Base Model:** RoBERTa-large (355M parameters)
2. **RADAR Model:** TrustSafeAI/RADAR-Vicuna-7B
3. **Input Processing:** Maximum sequence length of 512 tokens

The detection score is:

$$P(\text{AI-generated}) = \exp(\log\text{-softmax}(\text{logits})_0) \tag{12}$$

E.4 Model-Based Detectors Setup

Model Selection: Claude-3.5-Haiku, DeepSeek-V2.5, Gemma-3-27B, Llama-4-Scout-17B, Mistral-Nemo, Llama-4-Maverick-17B, WizardLM-2-8x22B

Prompt Engineering Strategy:

1. **System Message:** Establishes expert persona and task context
2. **Few-Shot Examples:** 3–4 carefully crafted demonstrations per task
3. **Multi-Turn Conversation:** Maintains context across examples
4. **Task-Specific Expertise:** Specialized personas for different detection tasks

Single-Sentence Classification Prompt Templates: **Task 1 — Paraphrase Source Attribution:**

> *"You are an expert text analyst specializing in paraphrase detection. Your task is to determine whether a paraphrased sentence was created by a human or by an AI/LLM system.*
> *CLASSIFICATION CRITERIA: - Human paraphrase (0): Natural flow, contextual understanding, human-like word choices - LLM paraphrase (1): Systematic rewording patterns, over-precision, AI-like formalization*
> *IMPORTANT: You must respond with ONLY the number 0 or 1."*

Task 2 — General Text Authorship Detection:

> *"You are an expert in AI-generated text detection. Determine whether a sentence was originally written by a human or generated by an AI/LLM system.*
> *CLASSIFICATION CRITERIA: - Human original (0): Authentic voice, natural imperfections, personal style - LLM generated (1): Training data patterns, generic expressions, artificial smoothness*
> *IMPORTANT: You must respond with ONLY the number 0 or 1."*

Task 3 — AI Text Laundering Detection:

> *"You are a specialist in detecting AI text laundering techniques. Determine the level of AI processing in text — distinguishing LLM-paraphrased original content versus LLM-paraphrased generated content.*
> *CLASSIFICATION CRITERIA: - LLM paraphrased original (0): AI paraphrase of human content - LLM paraphrased generated (1): AI paraphrase of AI content — multiple layers of AI processing*
> *IMPORTANT: You must respond with ONLY the number 0 or 1."*

Task 4 — Iterative Paraphrase Depth Detection:

> *"You are an expert in detecting iterative AI processing depth. Determine whether text has undergone fewer or more iterations of LLM paraphrasing.*
> *CLASSIFICATION CRITERIA: - 1st iteration paraphrase (0): Less deep AI processing - 3rd iteration paraphrase (1): Deeper AI processing, heavily transformed*
> *IMPORTANT: You must respond with ONLY the number 0 or 1."*

Task 5 — Deep Paraphrase Attack Detection:

> *"You are a cybersecurity expert specializing in detecting sophisticated AI paraphrase attacks. Distinguish between authentic human-written text and heavily processed AI paraphrases.*
> *CLASSIFICATION CRITERIA: - Human original (0): Authentic human expression, natural imperfections - Deep paraphrase attack (1): Heavily processed AI text, multiple transformation layers*
> *IMPORTANT: You must respond with ONLY the number 0 or 1."*

Sentence-Pair Comparative Prompting: For sentence-pair tasks, the prompting strategy shifts to comparative analysis where models must determine which sentence in a pair exhibits more human-like or AI-like characteristics. Corresponding comparative versions of the above system messages are used for each task, instructing the model to respond with 0 if the first sentence is more human-like, and 1 if the second sentence is more human-like.

Multi-Turn Conversation Structure:

Each evaluation follows a consistent multi-turn conversation pattern:

1. **System Message:** Task-specific expert persona and classification criteria
2. **Few-Shot Example 1–3:** User query with example text → Assistant response with label
3. **Target Query:** User query with actual text to classify → Assistant response (evaluated)

F Ethics Statements and Limitations

Ethics Statement Only publicly available datasets and pre-trained models are used in this study, accessed and utilized strictly for research purposes. The use of these resources complies with their original licenses and terms of access. No personally identifiable or sensitive information is present in any of the data used.

The code will be released under the MIT license to support transparency and reproducibility.

Limitations While PADBen provides comprehensive evaluation of paraphrase attack robustness, several aspects could be enhanced in future iterations:

Experimental Controls. The Experiment 2 trajectory analysis could benefit from stricter variable control, particularly maintaining consistent text length across paraphrasing iterations and expanding sample sizes beyond 100 texts per category.

Taxonomy Coverage. The five-type taxonomy focuses on core paraphrase attack scenarios but could expand to include additional variants, specifically extending Type 5 beyond 3 iterations and introducing intermediate iteratively-paraphrased human texts variants.

Detector Optimization. Zero-shot detectors are evaluated using default configurations without fine-tuning on PADBen data. While this approach assesses out-of-the-box robustness, adapted implementations could potentially improve performance.

Evaluation Comprehensiveness. Current sentence-pair tasks exclusively include paraphrased text in at least one position. Incorporating additional pairs without paraphrasing would provide more diverse evaluation scenarios.

These limitations represent opportunities for enhancement rather than fundamental flaws. PADBen's current design prioritizes realistic attack scenarios, mechanistic insights, and comprehensive detector evaluation within practical resource constraints.

References

1. Bill Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In *Third International Workshop on Paraphrasing (IWP2005)*. Asia Federation of Natural Language Processing, January 2005.
2. Hiu Ting Lau and Arkaitz Zubiaga. Understanding the effects of human-written paraphrases in llm-generated text detection, 2024.
3. Yuan Zhang, Jason Baldridge, and Luheng He. Paws: Paraphrase adversaries from word scrambling. In *Proceedings of NAACL*, 2019.
4. Kalpesh Krishna, Yixiao Song, Marzena Karpinska, John Wieting, and Mohit Iyyer. Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense, 2023.
5. mradermacher. Llama-3.1-8b-paraphrase-type-generation-apty-sigmoid-gguf. `https://huggingface.co/mradermacher/Llama-3.1-8B-paraphrase-type-generation-apty-sigmoid-GGUF`, 2024. Hugging Face.